

ATLASX: Full Report

Supervised by: DR/ Mohamed El-Kholy

Student's Name	Student's ID
Rodaina Mohamed Mustafa	23101892
Sama Osama Gomaa	23101571
Abdelrahman Mohamed	23101577

ATLASX: Frontend Report

A) Overview of our project:

ATLASX is a fast travel discovery app that lets you instantly explore any major city's top monuments, landmarks, and hidden gems, simply by typing the city name.

Instead of searching through dozens of websites or travel guides, you just open ATLASX, type "Paris", "Tokyo", "Rome", "Cairo"... and in less than a second you get a stunning visual gallery of the city's must-visit places with high-quality photos, descriptions, and categories (Landmark, Historic, Museum, Nature, etc.).

When you click on any place, a detailed modal opens with a large image and options to:

- **Mark as Visited (with confetti celebration)**
- **Add to Wishlist**
- **Like it**

All your choices are saved locally on your device, so you can build your personal travel map and dream list over time.

Technologies used:

Category	Technology used
Frontend Framework	React.js
Languages	JavaScript
Styling	Custom CSS3
3D-Animations	OGL.js (WebGL)
Icons	Bootstrap icons
State persistence	localStorage + custom useLocalStorage hook
Effects	Canvas-Confetti

Before we start we must define multiple words to actually understand **ATLASX**:

- 1. React:** JavaScript library for building user interfaces using components
- 2. Components:** Reusable, independent piece of UI (like a custom HTML tag)
- 3. Hooks:** Functions that let function components use state and lifecycle features

B) Component-by-Component Explanation

1. BackToTop.js

This React component implements a "Back to Top" button that enhances user experience on long-scrolling pages.

The button remains hidden initially and only becomes visible when the user scrolls more than 400 pixels down the page, at which point it smoothly fades into view. When clicked, it triggers a smooth scrolling animation that returns the user to the top of the page.

The component efficiently manages scroll event listeners to track the user's position and automatically cleans them up when unmounted, ensuring optimal performance without memory leaks.

This creates an intuitive navigation aid that helps users quickly return to the top of content-rich pages without manual scrolling.

```
1 import React, { useState, useEffect } from 'react';
2
3 const BackToTop = () => {
4   const [isVisible, setIsVisible] = useState(false);
5
6   useEffect(() => {
7     const handleScroll = () => {
8       setIsVisible(window.scrollY > 400);
9     };
10
11    window.addEventListener('scroll', handleScroll);
12    return () => window.removeEventListener('scroll', handleScroll);
13  }, []);
14
15  const scrollToTop = () => {
16    window.scrollTo({ top: 0, behavior: 'smooth' });
17  };
18
19  return (
20    <button
21      className={`back-to-top ${isVisible ? 'show' : ''}`}
22      onClick={scrollToTop}
23    >
24      ↑
25    </button>
26  );
27 };
28
29 export default BackToTop;
```

2. Hero.js

This React component creates a compact hero section that serves as an engaging introduction to a travel website.

It features a concise headline inviting users to "Discover Must-Visit Places," followed by a brief description highlighting monuments, hidden gems, and scenic views.

The section includes a simple call-to-action button labeled "Explore" that directs users to the search functionality. Despite its minimal code, this component effectively captures attention and encourages immediate user interaction with the platform's core offering of destination discovery.

```
1 import React from 'react';
2
3 const Hero = () => {
4   return (
5     <section className="hero" id="top">
6       <div className="hero-content">
7         <h1>Discover the World's Must-Visit Places</h1>
8         <p>Monuments, hidden gems & breathtaking views</p>
9         <a href="#search" className="btn btn-light">Start Exploring</a>
10       </div>
11     </section>
12   );
13 };
14
15 export default Hero;
```

3. Chatbot.js

This React component creates an interactive travel chatbot that helps users discover personalized destination recommendations through a conversational interface.

The chatbot guides users through a two-step conversation flow: first asking for their desired travel destination from a predefined list of cities, then inquiring about the type of atmosphere or "vibes" they're seeking. Using a sophisticated keyword matching system, the bot analyzes user preferences against categories like romantic, historic, modern, nature, adventure, and others to generate tailored place recommendations.

The component features a toggleable chat interface with smooth scrolling, message history, and visual recommendation cards displaying images and descriptions of suggested locations.

After providing recommendations, the chatbot automatically resets to help users explore new destinations, creating an engaging and intuitive travel planning experience that mimics a conversation with a knowledgeable travel assistant.

```
1 import React, { useState, useRef, useEffect } from 'react';
2 import { places } from '../data/placesData';
3
4 const Chatbot = () => {
5   const [isOpen, setIsOpen] = useState(false);
6   const [messages, setMessages] = useState([
7     {
8       type: 'bot',
9       text: "Hi! I'm your travel assistant. Where would you like to go and what kind of vibes are you looking for?"
10    }
11  ]);
12   const [userInput, setUserInput] = useState('');
13   const [conversationStep, setConversationStep] = useState('destination');
14   const [userData, setUserData] = useState({
15     destination: '',
16     vibes: ''
17   });
18   const messagesEndRef = useRef(null);
19
20   const scrollToBottom = () => {
21     messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
22   };
23
24   useEffect(() => {
25     scrollToBottom();
26   }, [messages]);
27
28   const cities = ['paris', 'tokyo', 'cairo', 'london', 'rome', 'newyork'];
29   const vibeKeywords = {
30     romantic: ['romantic', 'love', 'couple', 'intimate', 'charming'],
31     historic: ['historic', 'ancient', 'old', 'traditional', 'cultural', 'heritage'],
32     modern: ['modern', 'contemporary', 'urban', 'city', 'futuristic'],
```

4. Toast.js

This React component creates a simple and conditional toast notification system that displays temporary messages to users.

The component only renders when the “show” prop is true, otherwise it returns nothing, making it efficient and non-intrusive. When visible, it displays a clean notification container with the provided message content.

This lightweight toast system is ideal for showing brief alerts, success messages, or status updates that automatically disappear when the `show` prop becomes false, providing users with immediate feedback without permanently occupying screen space.

```
1 import React from 'react';
2
3 const Toast = ({ show, message }) => {
4   if (!show) return null;
5
6   return (
7     <div className="toast show">
8       <div className="toast-content">
9         <div className="toast-body">
10           {message}
11         </div>
12       </div>
13     </div>
14   );
15 };
16
17 export default Toast;
```

5. Navbar.js

This React component creates a dynamic navigation bar that features theme switching and user profile management capabilities.

The navbar displays a brand logo "ATLASX" and includes two interactive buttons: one for toggling between dark and light modes with appropriate icons, and another for accessing the user profile that shows a notification badge.

The badge dynamically calculates the total count of user interactions by combining visited places, wishlist items, and likes from localStorage, updating in real-time through event listeners and periodic checks.

This smart navigation system provides users with quick access to theme preferences and personal activity tracking while maintaining an elegant and functional interface.

```
1 import React from 'react';
2
3 const Navbar = ({ darkMode, setDarkMode, setShowProfile }) => {
4   const [badgeTotal, setBadgeTotal] = React.useState(0);
5
6   const calculateBadge = () => {
7     try {
8       const visited = JSON.parse(localStorage.getItem('visited') || '{}');
9       const wishlist = JSON.parse(localStorage.getItem('wishlist') || '{}');
10      const liked = JSON.parse(localStorage.getItem('liked') || '{}');
11
12      const visitedCount = Object.keys(visited).length;
13      const wishlistCount = Object.keys(wishlist).length;
14      const likesCount = Object.values(liked).reduce((a, b) => a + (b || 0), 0);
15
16      setBadgeTotal(visitedCount + wishlistCount + likesCount);
17    } catch (error) {
18      console.error('Error calculating badge:', error);
19    }
20  };
21
22  React.useEffect(() => {
23    calculateBadge();
24
25    const handleStorageUpdate = () => {
26      calculateBadge();
27    };
28
29    window.addEventListener('localStorageUpdate', handleStorageUpdate);
30    window.addEventListener('storage', handleStorageUpdate);
31
32    const interval = setInterval(calculateBadge, 1000);
```

6. PlaceCards.js

This React component creates an interactive place card that displays destination information with user engagement features.

Each card shows a location's image, category tag, name, and description, while incorporating personalized user indicators including visited status, wishlist inclusion, and like counts retrieved from localStorage.

The component implements intelligent image handling with loading skeletons, error fallbacks using SVG placeholders, and smooth fade-in animations. An overlay with a "View Details" prompt appears on hover, encouraging user interaction.

The card also features staggered entrance animations and comprehensive visual feedback that reflects user preferences, creating an engaging and personalized display for exploring travel destinations.

```
1 import React, { useState } from 'react';
2
3 const PlaceCard = ({ place, city, index, onClick }) => {
4   const [visited, setVisited] = React.useState(false);
5   const [wishlist, setWishlist] = React.useState(false);
6   const [liked, setLiked] = React.useState(false);
7   const [imageLoaded, setImageLoaded] = useState(false);
8   const [imageError, setImageError] = useState(false);
9
10  React.useEffect(() => {
11    const savedVisited = JSON.parse(localStorage.getItem('visited') || '{}');
12    const savedWishlist = JSON.parse(localStorage.getItem('wishlist') || '{}');
13    const savedLiked = JSON.parse(localStorage.getItem('liked') || '{}');
14
15    setVisited(savedVisited);
16    setWishlist(savedWishlist);
17    setLiked(savedLiked);
18  }, []);
19
20  const id = `${city}-${place.name}`;
21  const isVisited = visited[id];
22  const isWishlist = wishlist[id];
23  const likeCount = liked[id] || 0;
24
25  const placeholderSVG = "data:image/svg+xml;utf8," + encodeURIComponent(
26    '<svg xmlns="http://www.w3.org/2000/svg" width="800" height="450" viewBox="0 0 800 450">' +
27    '<rect width="100%" height="100%" fill="#f6f6f7"/>' +
28    '<text x="50%" y="50%" dominant-baseline="middle" text-anchor="middle" fill="black">' +
29    'Placeholder Image</text>' +
30    '</svg>'
31  );
```

7. PlaceModal.js

This React component creates a detailed modal overlay that displays comprehensive information about a travel destination when users select a place card.

The modal showcases a large image of the location along with its category tag, name, and description, while providing three key interactive features: marking places as visited (which triggers celebratory confetti), adding to a wishlist, and liking the destination.

All user interactions are persistently stored using localStorage and trigger real-time updates across the application.

The modal includes visual feedback through changing button states and toast notifications, creating an engaging and personalized experience for users to track their travel preferences and accomplishments.

```
1 import React from 'react';
2 import { useLocalStorage } from '../hooks/useLocalStorage';
3
4 const PlaceModal = ({ place, city, isOpen, onClose, showToast }) => {
5   const [visited, setVisited] = useLocalStorage('visited', {});
6   const [wishlist, setWishlist] = useLocalStorage('wishlist', {});
7   const [liked, setLiked] = useLocalStorage('liked', {});
8
9   if (!isOpen || !place) return null;
10
11   const id = `${city}-${place.name}`;
12   const isPlaceVisited = visited[id];
13   const isPlaceWishlist = wishlist[id];
14   const placeLikeCount = liked[id] || 0;
15
16   const triggerStorageUpdate = () => {
17     const event = new Event('localStorageUpdate');
18     window.dispatchEvent(event);
19   };
20
21   const handleVisited = () => {
22     const newVisited = { ...visited, [id]: true };
23     setVisited(newVisited);
24     showToast('Marked as visited! Congrats!');
25     triggerStorageUpdate();
26
27     if (typeof window.confetti === 'function') {
28       window.confetti({
29         particleCount: 100,
30         spread: 70,
31         origin: { y: 0.6 }
```

8. ProfileOfCanvas.js

This React component creates a sliding profile panel that displays comprehensive user travel statistics and activity history.

The offcanvas sidebar shows personalized metrics including visited places count, wishlist items, and total likes, along with a level progression system that increases based on travel activity.

It features a real-time recent activity feed showing the user's latest interactions with destinations, and includes a data reset option for clearing all stored preferences.

The component continuously monitors localStorage for updates through event listeners and periodic checks, ensuring the profile always reflects current user engagement while providing an overview of travel accomplishments and personalized insights.

```
1 import React from 'react';
2
3 const ProfileOfCanvas = ({ show, setShow }) => {
4   const [stats, setStats] = React.useState({
5     visited: 0,
6     wishlist: 0,
7     likes: 0
8   });
9   const [recentActivity, setRecentActivity] = React.useState([]);
10
11   const calculateStats = () => {
12     try {
13       const visited = JSON.parse(localStorage.getItem('visited') || '{}');
14       const wishlist = JSON.parse(localStorage.getItem('wishlist') || '{}');
15       const liked = JSON.parse(localStorage.getItem('liked') || '{}');
16
17       const visitedCount = Object.keys(visited).length;
18       const wishlistCount = Object.keys(wishlist).length;
19       const likesCount = Object.values(liked).reduce((a, b) => a + (b || 0), 0);
20
21       setStats({
22         visited: visitedCount,
23         wishlist: wishlistCount,
24         likes: likesCount
25       });
26
27       const activities = [];
28       Object.keys(visited).forEach(key => {
29         activities.push({ type: 'visited', key, timestamp: Date.now() });
30       });
31       Object.keys(wishlist).forEach(key => {
32         activities.push({ type: 'wishlist', key, timestamp: Date.now() });
```


9. Search.js

This React component creates a comprehensive search and discovery interface for exploring travel destinations.

It features a search form that allows users to find attractions in various cities, displaying results in an animated grid of interactive place cards.

The component manages loading states with visual feedback, handles city validation, and provides informative toast notifications. When users click on place cards, it opens detailed modals with additional information and interaction options.

The interface also includes a promotional "Why ATLASX?" section highlighting key features like handpicked places, offline functionality, and personalized journey tracking, creating a seamless user experience for discovering and engaging with travel destinations.

```
1 import React, { useState } from 'react';
2 import PlaceCard from './PlaceCard';
3 import PlaceModal from './PlaceModal';
4 import { places } from '../data/placesData';
5
6 const Search = ({ showToast }) => {
7   const [searchCity, setSearchCity] = useState('');
8   const [currentPlaces, setCurrentPlaces] = useState(places.paris);
9   const [currentCity, setCurrentCity] = useState('paris');
10  const [selectedPlace, setSelectedPlace] = useState(null);
11  const [showModal, setShowModal] = useState(false);
12  const [isLoading, setIsLoading] = useState(false);
13
14  const handleSearch = (e) => {
15    e.preventDefault();
16    if (!searchCity.trim()) return;
17
18    setIsLoading(true);
19    const cityKey = searchCity.toLowerCase().replace(/s+/g, '');
20    const cityPlaces = places[cityKey];
21
22    setTimeout(() => {
23      if (cityPlaces) {
24        setCurrentPlaces(cityPlaces);
25        setCurrentCity(cityKey);
26        showToast(`Found ${cityPlaces.length} places in ${searchCity}!`);
27      } else {
28        showToast('City not found. Try: Paris, Tokyo, Cairo, Rome, London, or New York');
29      }
30      setIsLoading(false);
31      setSearchCity('');
32    }, 800);
33  };
34
```

10. SeasonalRecommendations.js

This React component creates an interactive seasonal travel recommendations section that helps users discover destinations based on the time of year.

It features a tabbed interface with four seasons, spring, summer, autumn, and winter, each showcasing three curated destinations with compelling imagery and reasons to visit.

When users click on a destination card, it triggers a callback to display specific places within that location, creating a seamless exploration flow.

The component combines engaging visual design with practical travel planning by highlighting seasonal attractions like cherry blossoms in spring, beach destinations in summer, fall foliage spots in autumn, and winter wonderlands, making it easy for travelers to find inspiration for their next seasonal adventure.

```
1 import React, { useState } from 'react';
2
3 const SeasonalRecommendations = ({ onPlaceClick }) => {
4   const [activeSeason, setActiveSeason] = useState('spring');
5
6   const seasonalRecommendations = [
7     spring: {
8       title: "Spring Blossoms 🌸",
9       description: "Perfect destinations for cherry blossoms and mild weather",
10      destinations: [
11        {
12          name: "Kyoto, Japan",
13          reason: "Cherry blossom season",
14          image: "https://images.unsplash.com/photo-1493976040374-85c8e12f0c0e?auto=format&fit=crop",
15          place: { name: "Senso-ji Temple", city: "tokyo" }
16        },
17        {
18          name: "Amsterdam, Netherlands",
19          reason: "Tulip fields in bloom",
20          image: "https://images.unsplash.com/photo-1512470876302-972faa2aa9a4?auto=format&fit=crop",
21          place: { name: "Keukenhof Gardens", city: "amsterdam" }
22        },
23        {
24          name: "Washington D.C., USA",
25          reason: "National Cherry Blossom Festival",
26          image: "https://encrypted-tbn2.gstatic.com/licensed-image?q=tbn:ANd9GcSsRzmYld1Eg",
27          place: { name: "National Mall", city: "washington" }
28        }
29      ]
30    },
31     summer: {
32       title: "Summer Adventures 🌞",
33
```

11.TravelStats.js

This React component creates an engaging travel statistics section that showcases platform metrics and trending destinations.

It features a visually striking background using an Aurora component with dynamic color effects, complemented by a grid of four key statistics cards displaying impressive numbers like 1.2 million+ travelers helped and 500+ places listed. Below the main metrics, a trending destinations list ranks popular cities with visitor counts and growth trends, using visual indicators to highlight positive performance.

The component effectively builds credibility and inspires wanderlust by presenting compelling data about the platform's reach and current travel hotspots in an aesthetically pleasing, easy-to-digest format.

```
1 import React from 'react';
2 import Aurora from './Aurora';
3
4 const TravelStats = () => {
5   const stats = {
6     totalVisitors: "1.2M+",
7     countriesCovered: "50+",
8     placesListed: "500+",
9     userReviews: "45K+"
10  };
11
12  const popularDestinations = [
13    { name: "Paris", visitors: "250K", trend: "+15%" },
14    { name: "Tokyo", visitors: "180K", trend: "+22%" },
15    { name: "Rome", visitors: "165K", trend: "+8%" },
16    { name: "London", visitors: "155K", trend: "+12%" }
17  ];
18
19  return (
20    <section id="travel-stats" className="travel-stats-section">
21      <Aurora
22        colorStops={[ "#3A29FF", "#FF94B4", "#FF3232" ]}
23        blend={0.5}
24        amplitude={1.0}
25        speed={0.5}
26      />
27
28      <div className="container">
29        <div className="section-header">
30          <h2>Travel Statistics</h2>
31          <p>See what's trending in the world of travel</p>
32        </div>
33      </div>
34    </section>
35  );
36}
```

12.TravelStories.js

This React component creates an engaging travel stories section featuring firsthand accounts from travelers around the world.

It displays a grid of story cards with compelling images, author information, and location details, initially showing three stories with an option to expand.

Each story includes interactive buttons to read the full content in a modal overlay or directly explore the related destination.

The component maintains a clean, magazine-like layout while providing seamless integration with the main application's navigation system, allowing users to transition from inspirational stories to practical destination exploration with a single click.

```
1 import React, { useState } from 'react';
2
3 const TravelStories = ({ onPlaceClick }) => {
4   const [showAllStories, setShowAllStories] = useState(false);
5
6   const stories = [
7     {
8       id: 1,
9       title: "Sunset at the Eiffel Tower",
10      excerpt: "The most magical moment in Paris happens when the sun sets behind the Iron Lady...",
11      fullContent: "The Eiffel Tower at sunset is truly a sight to behold. As the golden hour approaches, the tower's iron lattice seemed to catch fire as the sun dipped below the horizon, and when darkness finally fell, the city lights began to twinkle like stars against the dark sky.",
12      author: "Sarah Chen",
13      location: "Paris, France",
14      readTime: "4 min read",
15      image: "https://images.unsplash.com/photo-1431274172761-fca41d930114?auto=format&fit=crop&w=600&q=80",
16      date: "2024-01-15",
17      relatedPlace: { name: "Eiffel Tower", city: "Paris" }
18     },
19     {
20       id: 2,
21       title: "Ancient Wonders of Cairo",
22       excerpt: "Walking through history at the Pyramids of Giza felt like stepping back in time...",
23       fullContent: "Standing before the Great Pyramid of Giza, I was overwhelmed by the sheer scale of human achievement. The best time to visit is early morning, before the crowds arrive and the heat becomes intense. Watching the sun rise over the desert landscape was a truly unforgettable experience.",
24       author: "Ahmed Hassan",
25       location: "Cairo, Egypt",
26       readTime: "6 min read",
27       image: "https://encrypted-tbn2.gstatic.com/licensed-image?q=tbn:ANd9GcSQnuQOVFPcqek7G68v36TyUceclDN"
28     }
29   ];
30
31   return (
32     <div>
33       <div>
34         <h2>Travel Stories</h2>
35         <p>Discover firsthand accounts from travelers around the world</p>
36       </div>
37       <div>
38         {stories.map((story) => (
39           <div>
40             <img alt={story.image} />
41             <div>
42               <h3>{story.title}</h3>
43               <p>{story.excerpt}</p>
44               <p>By {story.author} | {story.location}</p>
45               <p>{story.readTime}</p>
46             </div>
47             <div>
48               <button>Read Full Story</button>
49               <button>Explore {story.relatedPlace.name}</button>
50             </div>
51           </div>
52         ))}
53       </div>
54       <div>
55         <button>Show All Stories</button>
56       </div>
57     </div>
58   );
59}
```


13. WorldMap.js

This React component creates an interactive world map interface that allows users to explore travel destinations through visual geography.

The map features clickable city markers positioned over major global destinations like Paris, Tokyo, and New York, each represented with country flags.

When users select a city, the component triggers smooth animations and displays a preview of top local attractions below the map.

This interactive visualization provides an engaging, game-like experience for discovering new places, combining geographical context with practical destination information while maintaining seamless integration with the application's navigation system for deeper exploration of specific locations.

```
1 import React, { useState } from 'react';
2 import { places } from '../data/placesData';
3
4 const WorldMap = ({ onSelect, showToast, onPlaceClick }) => {
5   const [selectedCity, setSelectedCity] = useState(null);
6   const [isAnimating, setIsAnimating] = useState(false);
7
8   const cities = [
9     { id: 'paris', name: 'Paris', x: 48, y: 28, country: 'France', flag: 'FR' },
10    { id: 'london', name: 'London', x: 46, y: 25, country: 'UK', flag: 'GB' },
11    { id: 'rome', name: 'Rome', x: 50, y: 32, country: 'Italy', flag: 'IT' },
12    { id: 'cairo', name: 'Cairo', x: 55, y: 35, country: 'Egypt', flag: 'EG' },
13    { id: 'tokyo', name: 'Tokyo', x: 85, y: 32, country: 'Japan', flag: 'JP' },
14    { id: 'newyork', name: 'New York', x: 25, y: 30, country: 'USA', flag: 'US' }
15  ];
16
17   const handleCityClick = (city) => {
18     setIsAnimating(true);
19     setSelectedCity(city);
20
21     setTimeout(() => {
22       onSelect(city.id);
23       showToast(`Exploring ${city.name}!`);
24       setIsAnimating(false);
25     }, 800);
26   };
27
28   const handlePlaceClick = (place) => {
29     if (onPlaceClick) {
30       onPlaceClick(place, selectedCity.id);
31     }
32   };
33 }
```

14. Aurora.js

This React component creates a dynamic, animated aurora background effect using WebGL through the OGL library, and it's worth noting that this code is adapted from **ReactBits**, a collection of React patterns and components.

The Aurora component generates flowing, colorful gradient waves that simulate the natural aurora phenomenon, using custom shader programs with noise algorithms to create organic, undulating patterns.

It accepts customizable parameters for color gradients, animation speed, wave amplitude, and blend intensity, rendering the effect efficiently with WebGL for smooth performance.

The component automatically handles responsive resizing and clean resource disposal, making it suitable for use as an eye-catching background element in modern web applications.

```
1 import { Renderer, Program, Mesh, Color, Triangle } from 'ogl';
2 import { useEffect, useRef } from 'react';
3 import './Aurora.css';
4
5 const VERT = `#version 300 es
6 in vec2 position;
7 void main() {
8   gl_Position = vec4(position, 0.0, 1.0);
9 }
10 `;
11
12 const FRAG = `#version 300 es
13 precision highp float;
14
15 uniform float uTime;
16 uniform float uAmplitude;
17 uniform vec3 uColorStops[3];
18 uniform vec2 uResolution;
19 uniform float uBlend;
20
21 out vec4 fragColor;
22
23 vec3 permute(vec3 x) {
24   return mod(((x * 34.0) + 1.0) * x, 289.0);
25 }
26
27 float snoise(vec2 v){
28   const vec4 C = vec4(
29     0.211324865405187, 0.366025403784439,
30     -0.577350269189626, 0.024390243902439
31   );
32 }
```

15.App.js

This React component serves as the main application container that operate all the travel exploration website's features.

The App component manages global state including dark mode theme, user profile visibility, toast notifications, and modal displays, while providing keyboard shortcuts for enhanced user experience (Ctrl+K for search, Ctrl+D for theme toggle).

It seamlessly integrates all major sections navigation, hero banner, interactive world map, statistics, search functionality, seasonal recommendations, and travel stories and handles the communication between components through callback functions for city selection and place interactions.

The component ensures a cohesive user journey by maintaining consistent state across the application and providing smooth transitions between different exploration modes and detailed destination views.

```
1 import React, { useState, useEffect } from 'react';
2 import './styles/App.css';
3 import Navbar from './components/Navbar';
4 import Hero from './components/Hero';
5 import Search from './components/Search';
6 import WorldMap from './components/WorldMap';
7 import TravelStats from './components/TravelStats';
8 import TravelStories from './components/TravelStories';
9 import SeasonalRecommendations from './components/SeasonalRecommendations';
10 import ProfileOffcanvas from './components/ProfileOffcanvas';
11 import Toast from './components/Toast';
12 import BackToTop from './components/BackToTop';
13 import Chatbot from './components/Chatbot';
14 import PlaceModal from './components/PlaceModal';
15 import { places } from './data/placesData';
16
17 function App() {
18   const [darkMode, setDarkMode] = useState(false);
19   const [showProfile, setShowProfile] = useState(false);
20   const [toast, setToast] = useState({ show: false, message: '' });
21   const [currentCity, setCurrentCity] = useState('paris');
22   const [selectedPlace, setSelectedPlace] = useState(null);
23   const [showPlaceModal, setShowPlaceModal] = useState(false);
24
25   useEffect(() => {
26     const handleKeyPress = (e) => {
27       if ((e.ctrlKey || e.metaKey) && e.key === 'k') {
28         e.preventDefault();
29         const searchInput = document.querySelector('#search input');
30         if (searchInput) searchInput.focus();
31       }
32       if ((e.ctrlKey || e.metaKey) && e.key === 'd') {
```

C)Hooks Explanation

1.UseLocalStorage.js

This code implements a custom React hook called “useLocalStorage” that provides persistent state management using the browser's localStorage API.

The hook synchronizes React state with localStorage, automatically saving any value changes and initializing the state from stored data when the component mounts. It includes error handling for localStorage operations and dispatches custom events when updates occur, allowing other components to listen for changes. This reusable hook simplifies the process of creating persistent user preferences, cached data, or any other information that should survive browser sessions while maintaining the familiar interface of React's useState hook.

```
1 import { useState, useEffect } from 'react';
2
3 export const useLocalStorage = (key, initialValue) => {
4   const [value, setValue] = useState(() => {
5     try {
6       const item = window.localStorage.getItem(key);
7       return item ? JSON.parse(item) : initialValue;
8     } catch (error) {
9       console.error('Error reading localStorage key "${key}":', error);
10      return initialValue;
11    }
12  });
13
14   useEffect(() => {
15     try {
16       window.localStorage.setItem(key, JSON.stringify(value));
17       window.dispatchEvent(new Event('localStorageUpdate'));
18     } catch (error) {
19       console.error('Error setting localStorage key "${key}":', error);
20     }
21   }, [key, value]);
22
23   return [value, setValue];
24 };
```

D) Data Flow: How the App.js connects to all sections

The data flow in ATLASX follows a centralized operation model where App.js serves as the command center, managing all critical application state including theme preferences, user interface visibility, city selections, and place interactions.

Through the callback system, child components like **WorldMap**, **Search**, and **SeasonalRecommendations** communicate user actions upstream to **App.js**, which then processes these events and propagates state changes back down through props. This unidirectional data flow ensures consistent application behavior when a user selects a city on the **WorldMap**, **App.js** updates the current city state, automatically refreshing the Search component with relevant attractions while maintaining synchronization across all sections.

The architecture employs both direct prop drilling for immediate state sharing and a custom event system for **localStorage** synchronization, creating a responsive ecosystem where user interactions in one component seamlessly update dependent components without tight coupling, resulting in a cohesive and predictable user experience throughout the travel discovery journey.

E) Conclusion

ATLASX shows how modern web development can create engaging and user-friendly travel apps. By using React's component system and smart state management, the app makes exploring new places simple and enjoyable. The clean design and smooth interactions let users easily discover destinations and track their travel interests.

The project proves that good technical choices lead to great user experiences. Features like instant search, personalized saving options, and dark mode work together to make travel planning both fun and practical. **ATLASX** successfully combines useful functionality with an attractive interface, showing how thoughtful frontend development can build apps that people love to use.

ATLASX: Backend Report

A)What is Backend?

The backend of a web application refers to the server-side component responsible for managing data, processing requests, enforcing business logic, and handling interactions with databases or storage systems, all while remaining invisible to end-users who interact primarily with the frontend interface. It ensures security, scalability, and data persistence. APIs, or Application Programming Interfaces, are structured sets of rules and endpoints that allow different software components such as a frontend client and backend server, to communicate effectively; in this context, they are typically RESTful APIs using HTTP methods like GET, POST, PUT, and DELETE to expose backend functionalities for tasks like user authentication, data retrieval, and content management.

B)Component-by-Component Explanation

1)Chatbot.js

This code creates a recommendation system for travel destinations within a web application. It sets up an HTTP POST endpoint at `/recommend` that accepts a country and an array of vibes (like "relaxing" or "cultural") from the user's request. The server first validates this input and then converts everything to lowercase to ensure a case-insensitive search. It then filters through a master list of places, checking each one to see if it matches the specified country (or city) and contains at least one of the requested vibes. Finally, it sends back the top five matching places as a JSON response. In short, it's a smart travel filter that provides personalized destination suggestions based on location and mood

```
backend > routes > JS chatbotjs > ...
1  const express = require('express');
2  const { getPlaces } = require('../data/persistence');
3  const router = express.Router();
4  router.post('/recommend', (req, res) => {
5    const { country, vibes } = req.body;
6    if (!country || !vibes || !Array.isArray(vibes)) {
7      return res.status(400).json({ msg: 'Invalid input: country and vibes array required' });
8    }
9    const lowerCountry = country.toLowerCase().trim();
10   const lowerVibes = vibes.map(v => v.toLowerCase().trim());
11   const recommendations = getPlaces().filter(place => {
12     const matchesCountry =
13       place.country.toLowerCase().includes(lowerCountry) ||
14       place.city.toLowerCase().includes(lowerCountry);
15     const matchesVibe = lowerVibes.some(vibe =>
16       place.vibes.some(pv => pv.toLowerCase() === vibe)
17     );
18     return matchesCountry && matchesVibe;
19   });
20   res.json(recommendations.slice(0, 5));
21 });
22 module.exports = router;
```

2) auth.js

This code creates an Express authentication router that handles user **signup** and **login** using secure practices. When a user signs up, it first checks if the username already exists, then hashes the password with **bcrypt** before saving the user data (along with default arrays like likes and wishlist). After successful signup, it generates a **JWT token** that contains the user's ID and role and expires in 24 hours. For login, it looks up the user by username, compares the entered password with the stored hashed password, and if valid, issues a new

JWT token with the same info. The token allows the server to identify and authorize the user in future requests, while keeping passwords secure and never storing them in plain text.

```
backend > routes > JS auth.js > router.post('/login') callback
1  const express = require('express');
2  const jwt = require('jsonwebtoken');
3  const bcrypt = require('bcryptjs');
4  const { getUsers, addUser } = require('../data/persistence');
5  require('dotenv').config();
6  const router = express.Router();
7  router.post('/signup', async (req, res) => {
8    const { username, password, role = 'user' } = req.body;
9    if (getUsers().find(u => u.username === username)) {
10     return res.status(400).json({ msg: 'Username already exists' });
11   }
12   const hashedPassword = await bcrypt.hash(password, 10);
13   const newUser = {
14     id: Date.now().toString(),
15     username,
16     password: hashedPassword,
17     role,
18     likes: [],
19     wishlist: [],
20     visited: []
21   };
22   addUser(newUser);
23   const token = jwt.sign({ id: newUser.id, role: newUser.role }, process.env.JWT_SECRET || 'fallback_secret', { expiresIn: '24h' });
24   res.json({ token, role: newUser.role });
25 });
26 router.post('/login', async (req, res) => {
27   const { username, password } = req.body;
28   const user = getUsers().find(u => u.username === username);
29   if (!user || !(await bcrypt.compare(password, user.password))) {
30     return res.status(400).json({ msg: 'Invalid credentials' });
31   }
32 });
```

3) Places.js

This code defines an Express router for managing **places** with role-based security. Anyone can fetch all places using the GET / route, but creating, updating, or deleting a place is restricted to **admins only**. The adminAuth middleware checks for a JWT token in the Authorization header, verifies it using the secret key, and ensures the user's role is admin before allowing access. When an admin adds a place, a unique ID is generated and the place is saved; updating a place modifies its data if it exists, and deleting removes it by ID. If the token is missing, invalid, or the user is not an admin, the request is rejected, keeping sensitive actions protected.

```
backend > routes > JS places.js > router.put('/id') callback
1  const express = require('express');
2  const jwt = require('jsonwebtoken');
3  const { getPlaces, addPlace, updatePlace, deletePlace } = require('../data/persistence');
4  require('dotenv').config();
5  const router = express.Router();
6  const adminAuth = (req, res, next) => {
7    const token = req.header('Authorization')?.replace('Bearer ', '');
8    if (!token) return res.status(401).json({ msg: 'No token' });
9    try {
10     const decoded = jwt.verify(token, process.env.JWT_SECRET || 'fallback_secret');
11     if (decoded.role !== 'admin') return res.status(403).json({ msg: 'Admin access required' });
12     next();
13   } catch (err) {
14     res.status(401).json({ msg: 'Invalid token' });
15   }
16 };
17 router.get('/', (req, res) => {
18   res.json(getPlaces());
19 });
20 router.post('/', adminAuth, (req, res) => {
21   const newPlace = {
22     _id: Date.now().toString(),
23     ...req.body
24   };
25   addPlace(newPlace);
26   res.json(newPlace);
27 });
```

4) Stories.js

This code defines an Express router for handling **travel stories** with authentication and ownership rules. Anyone can view stories using GET /, but creating or deleting a story requires the user to be logged in with a valid **JWT token**. When a user posts a story, the server verifies the token, checks that all required fields are provided, then saves the story with a unique ID, the author's user ID, location, image, and timestamp. For deleting a story, the token is again verified, and the action is only allowed if the user is either an **admin** or the original author of the story; otherwise, access is denied. This setup ensures stories are public to read, but only authorized users can create or remove them.

```
backend > routes > JS stories.js > router.post('/') callback
1  const express = require('express');
2  const jwt = require('jsonwebtoken');
3  const { getStories, data, saveData } = require('../data/persistence');
4  require('dotenv').config();
5  const router = express.Router();
6  router.get('/', (req, res) => {
7    res.json(getStories());
8  });
9  router.post('/', (req, res) => {
10   const authHeader = req.header('Authorization');
11   if (!authHeader || !authHeader.startsWith('Bearer ')) {
12     return res.status(401).json({ msg: 'Please login to share a story' });
13   }
14   const token = authHeader.split(' ')[1];
15   try {
16     const decoded = jwt.verify(token, process.env.JWT_SECRET || 'fallback_secret');
17     const { title, excerpt, fullContent, location, image } = req.body;
18     if (!title || !excerpt || !fullContent || !location) {
19       return res.status(400).json({ msg: 'Please fill all required fields' });
20     }
  }
```

5) UserAction.js

This code creates an Express router that manages **user interactions** (likes, wishlist, and visited places) and protects all routes with JWT authentication. The auth middleware extracts and verifies the token, finds the corresponding user, and attaches the user data to the request. The GET / route returns the user's likes, wishlist, and visited places along with summary counts. The POST /like route lets a user like a place and increases the like count if it was liked before, while /wishlist and /visited add a place to their respective lists only once. Overall, this ensures only logged-in users can interact with places and keeps all actions tied to the authenticated user.

```
backend > routes > JS userActions.js > router.get('/') callback
1  const express = require('express');
2  const jwt = require('jsonwebtoken');
3  const { getUsers } = require('../data/persistence');
4  require('dotenv').config();
5  const router = express.Router();
6  const auth = (req, res, next) => {
7    const token = req.header('Authorization')?.replace('Bearer ', '');
8    if (!token) return res.status(401).json({ msg: 'No token' });
9    try {
10     const decoded = jwt.verify(token, process.env.JWT_SECRET || 'fallback_secret');
11     const user = getUsers().find(u => u.id === decoded.id);
12     if (!user) return res.status(401).json({ msg: 'User not found' });
13     req.user = user;
14     next();
15   } catch (err) {
16     res.status(401).json({ msg: 'Invalid token' });
17   }
18 };
19 router.get('/', auth, (req, res) => {
```


6) Data.json

This JSON file represents the **stored data for the application**. It contains three main sections: **users**, **places**, and **stories**. The users array includes an admin and a regular user, each with a unique ID, username, encrypted (hashed) password, role, and personal activity data like likes, wishlist, and visited places showing that the user *samsoma* has liked the Eiffel Tower once. The places array stores travel destinations with details such as name, description, image, location, tags, vibes, and best season to visit. The stories array is currently empty, meaning no users have shared travel stories yet. Overall, this file acts as a simple database that keeps all application data organized and persistent.

```

1  {
2    "users": [
3      {
4        "id": "admin1",
5        "username": "admin",
6        "password": "$2a$10$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi",
7        "role": "admin",
8        "likes": [],
9        "wishlist": [],
10       "visited": []
11     },
12     {
13       "id": "1766541552391",
14       "username": "samsoma",
15       "password": "$2a$10$6ECxCbRE0YMwwLE6hKjvPuZBBjPmDBBIatP0i6Q.YqpagNRdwg6y0",
16       "role": "user",
17       "likes": [
18         {
19           "placeId": "paris1",
20           "count": 1
21         }
22       ],
23       "wishlist": [],
24       "visited": []
25     }
26   ],

```

7) Persistence.js

This file acts as a **simple persistence layer (mini database)** for the app using a JSON file instead of a real database. It loads data from data.json when the server starts, or initializes default data (an admin user and some famous places) if the file doesn't exist or is corrupted. All data is kept in memory inside the data object, and any change like adding a user, place, or story, or updating/deleting a place is immediately saved back to the JSON file to make it persistent. The file also exposes helper functions to **get data** (users, places, stories) and **modify data** safely, so the rest of the app can read and update information without dealing directly with file handling

```

backend > data > JS persistence.js > load data
1  const fs = require('fs');
2  const path = require('path');
3
4  const DATA_FILE = path.join(__dirname, 'data.json');
5
6  let data = {
7    users: [],
8    places: [],
9    stories: []
10 };
11
12 function loadData() {
13   if (fs.existsSync(DATA_FILE)) {
14     try {
15       const fileContent = fs.readFileSync(DATA_FILE, 'utf-8');
16       const parsed = JSON.parse(fileContent);
17       data.users = parsed.users || [];
18       data.places = parsed.places || [];
19       data.stories = parsed.stories || [];
20       console.log('Persistent data loaded from data.json');
21     } catch (err) {
22       console.error('Error reading data.json, starting fresh', err);
23       initializeDefaultData();
24     }
25   } else {
26     initializeDefaultData();
27   }
28 }

```

8) Server.js

This code is the **main entry point of the backend server**. It starts by loading environment variables and calling `loadData()` to read persistent data from the JSON file into memory before the server runs. Then it creates an Express app, enables **CORS** for cross-origin requests, and sets up JSON body parsing. After that, it connects different API routes for authentication, places, stories, the chatbot, and user actions, each handling a specific part of the application logic. Finally, it starts the server on the specified port (default 5000) and logs helpful messages confirming the server is running, where data is stored, and the default admin login credentials.

```
backend > JS server.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const { loadData } = require('./data/persistence');
4  require('dotenv').config();
5  loadData();
6  const app = express();
7  app.use(cors());
8  app.use(express.json());
9  app.use('/api/auth', require('./routes/auth'));
10 app.use('/api/places', require('./routes/places'));
11 app.use('/api/stories', require('./routes/stories'));
12 app.use('/api/chatbot', require('./routes/chatbot'));
13 app.use('/api/userActions', require('./routes/userActions'));
14 const PORT = process.env.PORT || 5000;
15 app.listen(PORT, () => {
16   console.log(`Server running on port ${PORT}`);
17   console.log('Data persists in backend/data/data.json');
18   console.log('Admin login: username="admin", password="password"');
19 });
```

C) Data Flow

In this application, **data moves between the frontend and backend through HTTP requests and JSON responses** using a clear request–response cycle. The frontend (usually a React or web app) sends requests such as GET, POST, PUT, or DELETE to specific API endpoints like `/api/auth`, `/api/places`, or `/api/stories`. These requests may include data in the request body (for example, login credentials, a new place, or a story) and sometimes a JWT token in the Authorization header to prove the user is logged in. The backend receives the request through Express routes, validates the data and permissions, processes it using the persistence layer, and then sends back a JSON response containing either the requested data or a success/error message.

Internally, when the backend modifies data (adding a user, liking a place, posting a story), it updates the in-memory data object and immediately saves it to `data.json` so the changes persist. When the frontend later requests data again, the backend reads from this in-memory data (which was loaded from the file on startup) and returns the updated information. In short, the **frontend only talks to the backend via APIs**, the **backend controls all logic and security**, and the **JSON file acts as long-term storage**, ensuring data flows safely and consistently in both directions.

D) Conclusion

Overall, this backend is a **complete, well-structured REST API** that powers a travel application from end to end. It handles **authentication and authorization** using JWTs, securely managing users with hashed passwords and role-based access (admin vs user). It exposes clear routes for **places, stories, and user interactions** (likes, wishlist, visited), ensuring that public data is readable by everyone while sensitive actions are restricted to authenticated users or admins. Data persistence is achieved through a JSON-based storage layer that loads on server startup and saves every change, keeping the system simple yet reliable without a full database. The frontend communicates with the backend through JSON APIs, sending requests and receiving responses in a secure, controlled flow. Together, all these parts work as one cohesive system that ensures **security, data integrity, persistence, and clean separation of responsibilities**, making the application easy to understand, maintain, and extend.
